Software Architecture
and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 01

TH Aschaffenburg
university of applied sciences

Prof. Dr. Jóakim von Kistowski

Computer Science
Karlsruhe Institute of Technology

PhD Computer Science
University of Würzburg

Research Group Leader
University of Würzburg

Senior Software Architect / DevOps Team Lead
DATEV eG

Member, Chair, Contributor
Standard Performance Evaluation Corporation (SPEC)

TH Aschaffenburg
university of applied sciences

Research Topics

Looking for Bachelor/Master Thesis Topics?

GreenIT and Energy Efficiency

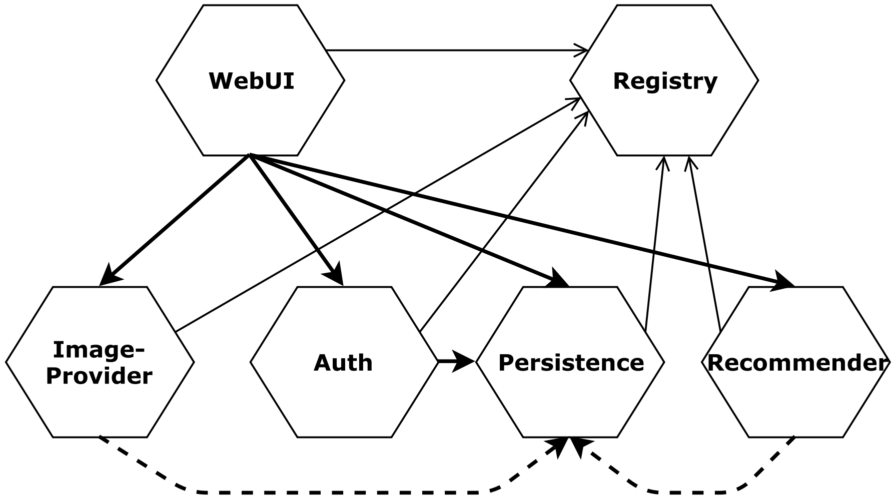
- Energy Efficiency of Servers
- Energy Efficiency of Software
- Green Software Operations
- Efficient Management of Server Fleets

Software Architecture and Quality

- Microservice Architectures
- Cloud-native Computing
- Software Resilience and Chaos Testing
- Software Performance Benchmarking and Analysis

Extracting Service Demands using Agentic AIs

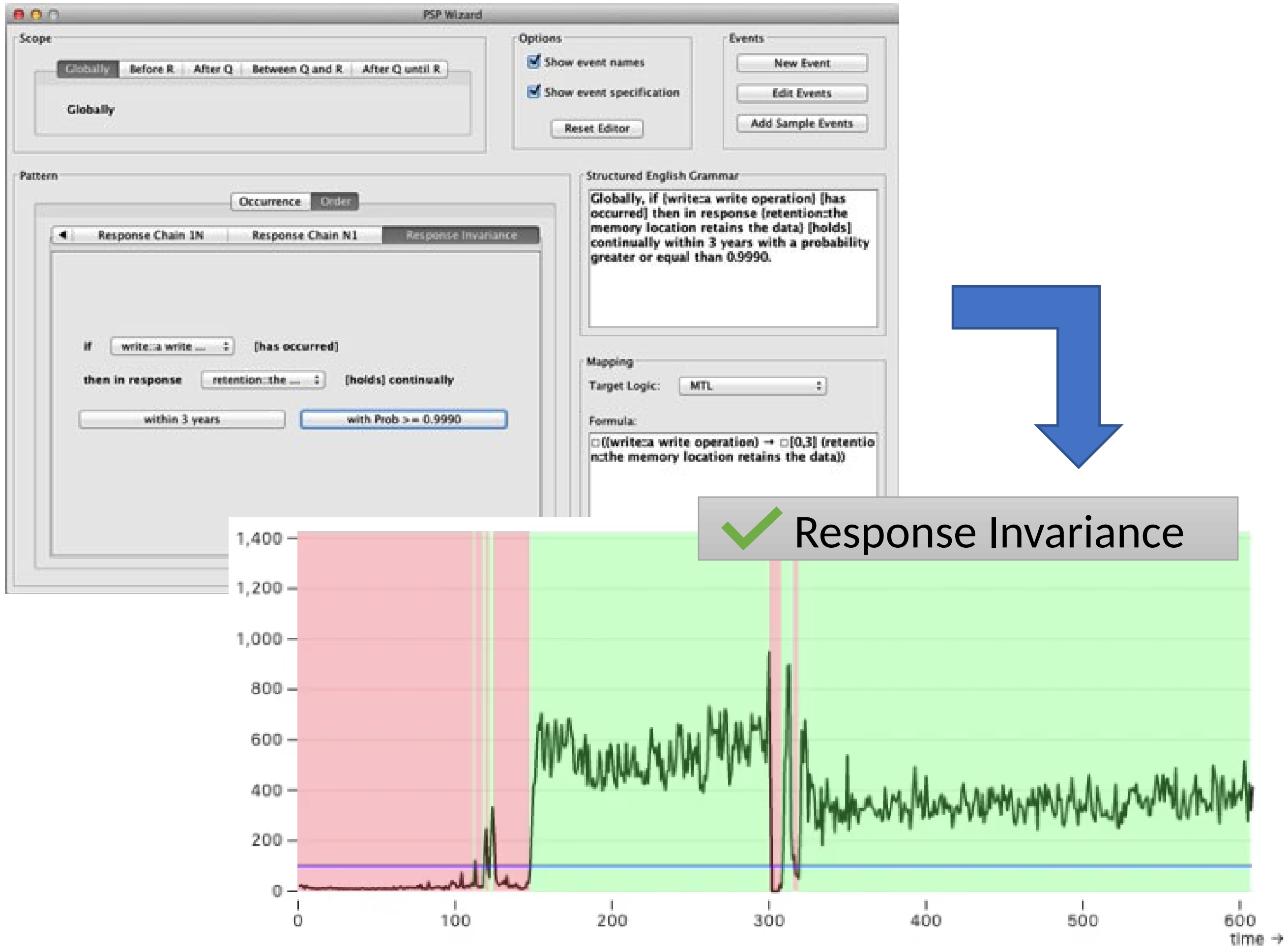
- Background:
    - Service Demand: Amount of Resources needed by an operation.
      E.g.: How much CPU does a single Log-in need on my server?
    - Knowledge about Service Demands enables DevOps-Experts to correctly configure and manage applications in datacenters or on end-user devices.
- Goals:
    - Autmated extraction of Service Demands using Agentic LLMs for an example Microservice Web-Application
    - Evaluate speed-up: How much work does using LLMs save when compared to traditional methods?



Contact: *joakim.vonkistowski@th-ab.de*

Explaining Property Specification Patterns Through Illustrative Examples

- Background:
  - Property Specification Patterns (PSP) provide a catalog of predefined, reusable, and structured templates for expressing requirements in formal languages
  - Example: The Recurrence Pattern describes that a property must hold repeatedly
- Goals:
  - Select and define example datasets that demonstrate both typical use cases and relevant edge cases for the different PSPs
  - Visualize these examples and integrate them into the existing „DiSpel“ tooling environment
  - Evaluate the usefulness of the examples in a quantitative user study



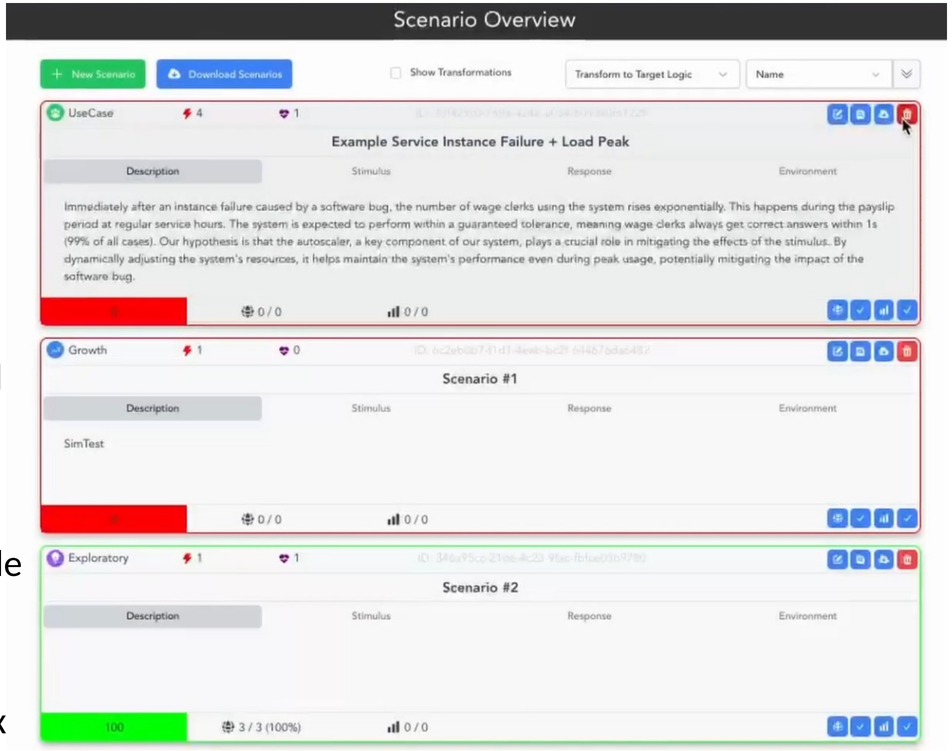
References:

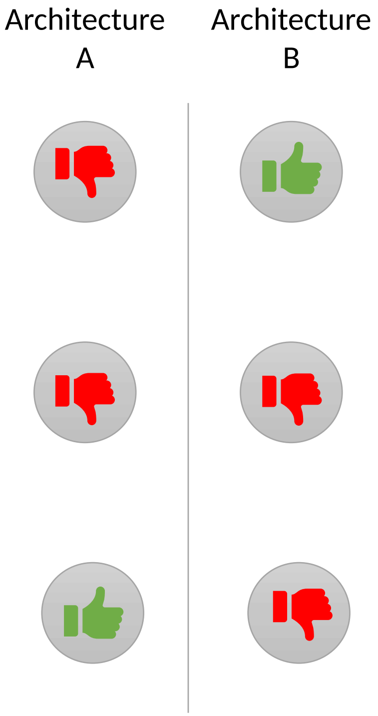
- Autili, Marco, et al. "Aligning qualitative, real-time, and probabilistic property specification patterns using a structured english grammar." *IEEE Transactions on Software Engineering* 41.7 (2015): 620-638.
- Frank, Sebastian, et al. "Dispel cockpit: Specification, verification, and refinement of resilience scenarios." *European Conference on Software Architecture*. Cham: Springer Nature Switzerland, 2024.

Contact: *joakim.vonkistowski@th-ab.de*

Extending DiSpel Cockpit for Scenario-based Analysis

- Background:
  - The Architecture Trade-off Analysis Method (ATAM) evaluates and compares software architectures through quality scenarios
  - DiSpel Cockpit supports analysis of resilience scenarios through monitoring and model-based simulation. However, its current capabilities for comparing architectural alternatives are limited
- Goals:
  - Extend the DiSPel Cockpit for analysis of multiple architectures and provide appropriate visualizations
  - Improve related functionality, such as support for multiple monitoring datasets, more complex scenarios, and more advanced mechanisms calculating scenario ratings
  - Evaluate the effectiveness and capability of extended approach





References:

- Bass, Len, Paul Clements, and Rick Kazman. "Software Architecture in Practice." (2013).
- Kazman, Rick, Mark Klein, and Paul Clements. "ATAM: Method for architecture evaluation." (2000).
- Frank, Sebastian, et al. "Dispel cockpit: Specification, verification, and refinement of resilience scenarios." *European Conference on Software Architecture*. Cham: Springer Nature Switzerland, 2024.

Contact: *joakim.vonkistowski@th-ab.de*

	Monday	Tuesday	Wednsday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00		Lecture Software Design International			Exercises Software Design International **Only on March 20**
	Lunch Break				
14:00 - 15:30		Exercises Software Design International			
15:45 - 17:15					
17:30 - 19:00					

TH Aschaffenburg
university of applied sciences

Lectures and Practical Exercises

Lectures

- Room: C2-225

- Totally interactive!

Practical Exercises

- Room: C2-225
- In small groups
- Bring your PC
- Pen and some paper helps

- Programming exercises and worksheets

On Public Holidays

SD Lectures are on Friday!
- Lots of public holidays on Friday

SDI Lectures are on Wednesday:
- Not so many public holidays on Wednesday
- Solution: Open Space and Live Coding Sessions on weeks with Friday off
- Open Spaces are announced ahead of time in Moodle
- First Open Space: April 1

TH Aschaffenburg
university of applied sciences

Moodle Course

German:
- SoSe26_SWA

International:
- SoSe26_SWA_SDI

- Enrollment Key: **SagaPattern_117**

Check-In, Part I

Recap:
What do you remember from
Collaboration, **Quality**, and Test**?**

Check-In, Part II

Recap:
What is a non-functional requirement?

Thanks and Acknowledgments

This lecture and the practical exercises are based a mix of original classes and the classes of

- † **Prof. Dr. André van Hoorn (University of Hamburg)**
- **Prof. Dr.-Ing. Samuel Kounev (University of Würzburg)**

TH Aschaffenburg
university of applied sciences

Introduction to Software Architecture

What is Software Architecture?

Very short Recap: Software Engineering

Outlook:
Macro-architecture vs. Micro-architecture

Discussion

What do you think Software Architecture is?

Software architecture is

- *“The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.” [IEEE standard 1471]*

- *“The structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.” [Bass2003]*

- *“A software system’s architecture is the set of principal design decisions made about the system.” [Taylor2008]*

Discussion

What does a Software Architect do?

Software Architect:

„a software engineer responsible for high-level design choices related to overall system structure and behavior“

[Software Engineering Institute. Carnegie Mellon University. 2022]

Which of these roles must a Software Architect fulfill? (1/2)

A Software architect is a/an

- Technical Expert
- Leader
- Domain Expert
- Controller
- Teacher
- Negotiator
- Consultant

Which of these roles must a Software Architect fulfill? (2/2)

A Software architect is a/an

- Technical Expert
- Leader
- ~~Domain Expert~~
- ~~Controller~~
- Teacher
- Negotiator
- Consultant

TH Aschaffenburg
university of applied sciences

Discussion

Do Software Architects shape business organizations?

Software Architecture and People

- Conway's Law

„Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.“

Melvin E. Conway

Human interactions, relations and organizational structures are a part of software architecture

Running Examples

Two Example-Systems with different quality requirements

E-Commerce: TeaStore

Web shop for Tea

Gaming: Online-Shooter

Online Multiplayer-Shooter for up to 64 players

Discussion: Effects of Conway’s Law

TeaStore

Online-Shooter

Assumption: TeaStore and Online-Shooter belong to the same company

- **What are the potential effects of this organizational structure on these two products?**

Assumption: Our company has a central authorization / authentication-platform/department for both products

- **What are the potential advantages or disadvantages for both products?**
- **What are potential central departments in this company that could affect both products?**

Software Architecture Methods

- Goal: Make Decisions or evaluate software architecture (**alternatives**)

- Focus on different aspects
  - Requirements
  - Quality-Requirements (Non-Functional Requirements)
  - Technical Framework
  - Organizational Framework

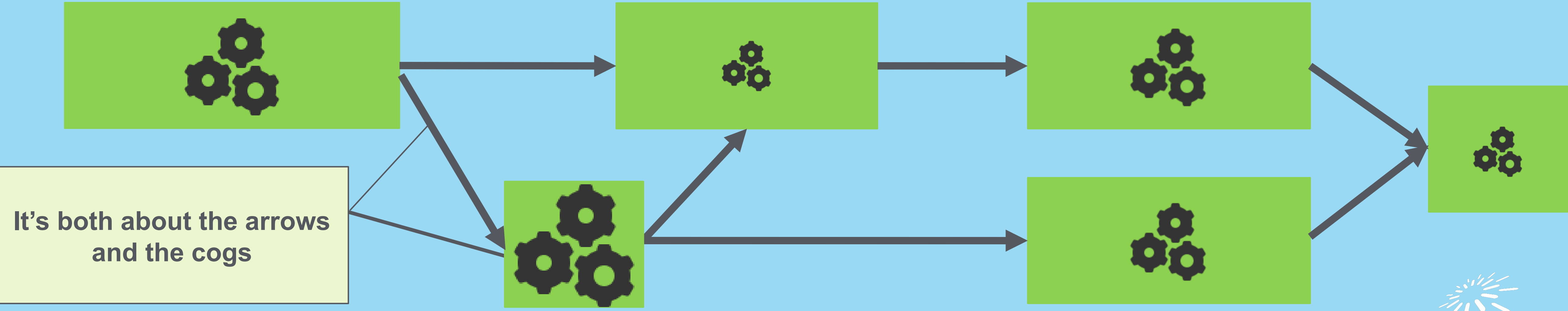
- May differ in granularity and detail, with which the different aspects are taken into consideration

Discussion

Why do we collect quality requirements for a software architecture?

Motivation for Quality Requirements

- A software architect defines the properties that can be achieved with sufficient quality
- e.g.: Performance <-> Extensibility <-> Security<-> Reusability

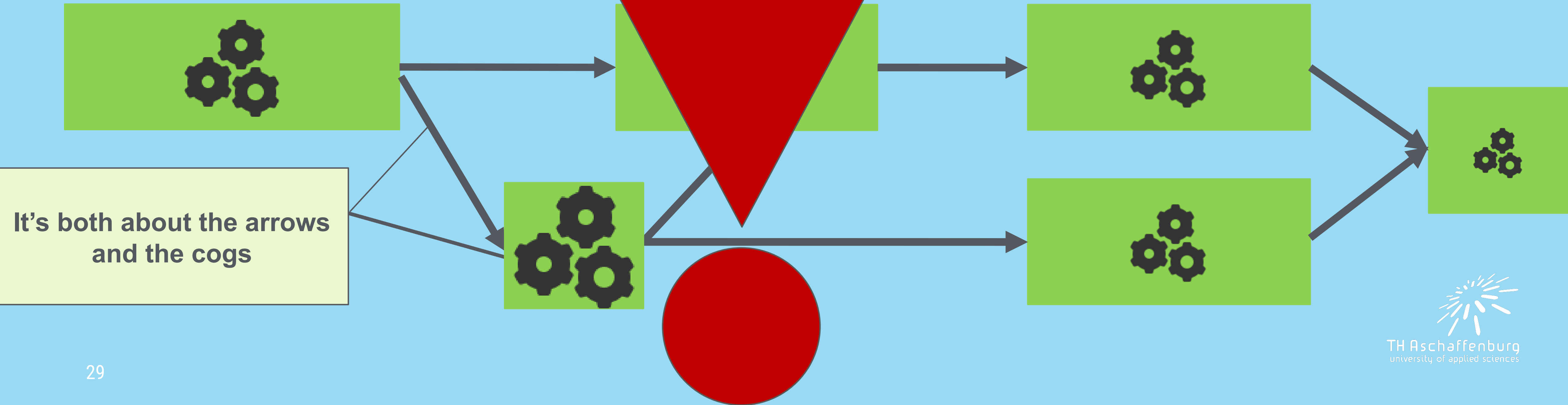


TH Aschaffenburg
university of applied sciences

Motivation for Quality Requirements

- A software architect define ... with sufficient quality
- e.g.: Performance <-> Extensibil ... ty

Software architectures are **neither correct nor wrong** without **quality requirements**!



TH Aschaffenburg
university of applied sciences

Question

Examples of Software that has clearly been designed with certain quality aspects in mind?

Introduction to Software Architecture

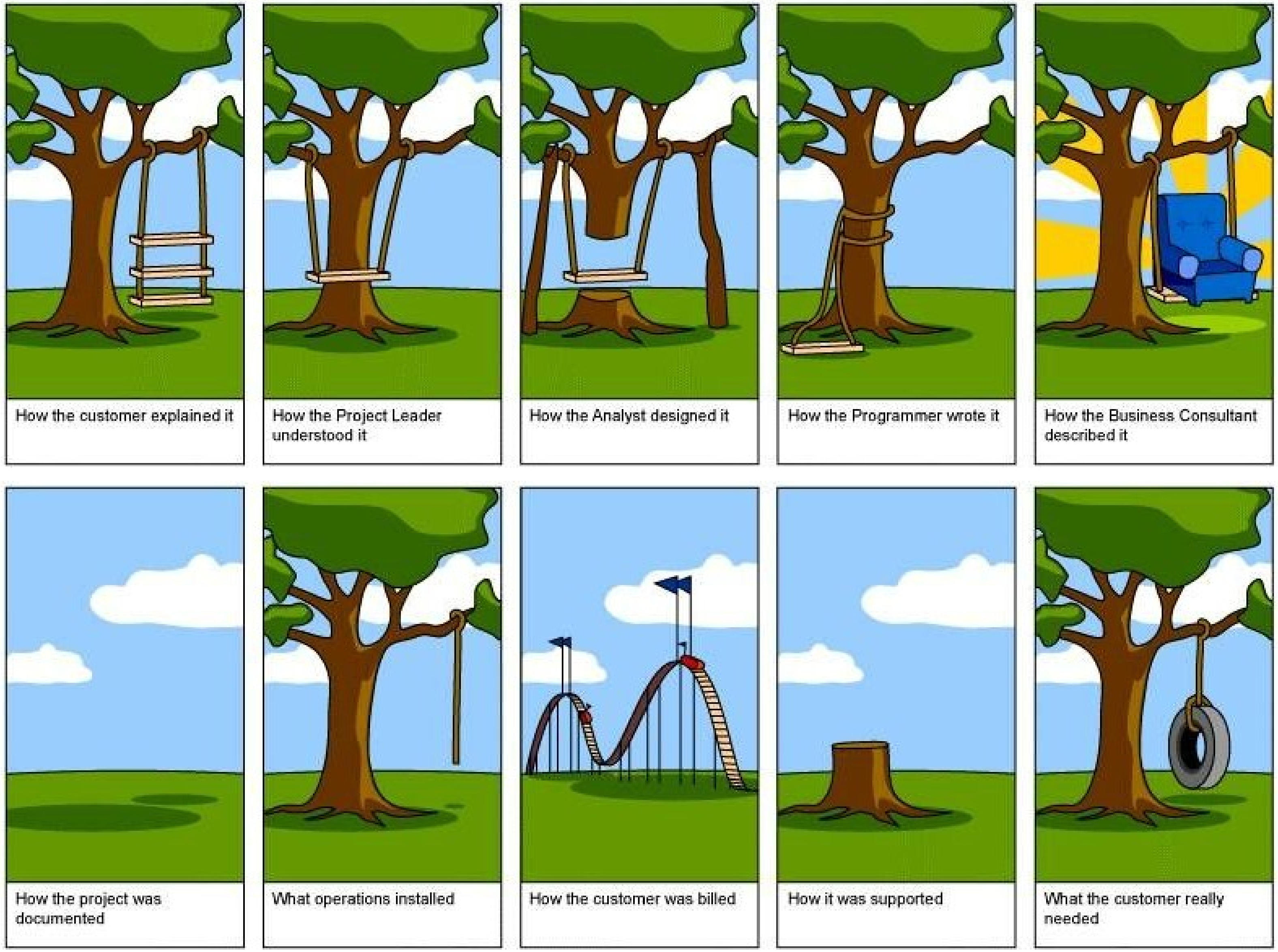
What is Software Architecture?

Very short Recap: Software Engineering

Outlook:
Macro-architecture vs. Micro-architecture

TH Aschaffenburg
university of applied sciences

Software Requirements



TH Aschaffenburg
university of applied sciences

Question

Why do we have to deal with software quality?

Question

What are the challenges of software quality assurance?

TH Aschaffenburg
university of applied sciences

Software Quality Assurance

Organizational

Organization

Analytic

Diagnosis

Constructive

Prevention

Quality cannot be tested „into a product“!

Organizational and constructive measures are significantly more important, they are supplemented by analytical measures.

Software Quality Assurance

Organizational	Analytic	Constructive
Organization	Diagnosis	Prevention
Software project management		constructive Software Engineering

Software Architecture

Where does Software Architecture fit?

Software Quality Assurance

Organizational

Organization

Analytic

Diagnosis

Constructive

Prevention

Software project management

constructive
Software Engineering

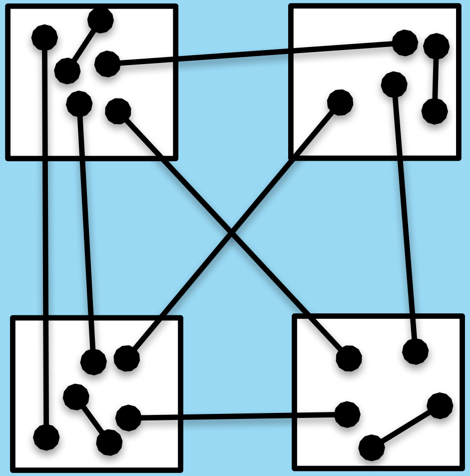
Software Architecture

TH Aschaffenburg
university of applied sciences

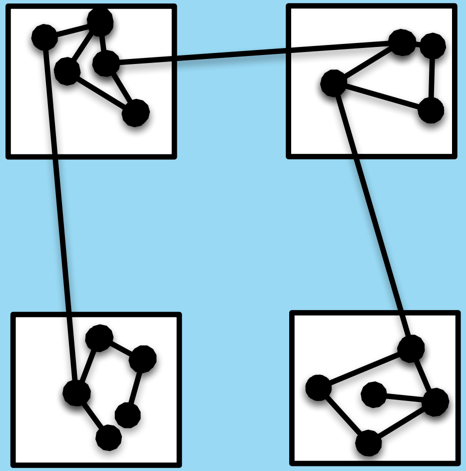
System Partitioning / Decomposition

Good architectures strive to achieve –

- cohesive responsibilities (***high cohesion*** within partitions)
- separation of concerns (***low coupling*** between partitions)



Low cohesion
High coupling



Low coupling
High cohesion

Introduction to Software Architecture

What is Software Architecture?

Very short Recap: Software Engineering

Outlook:
Macro-architecture vs. Micro-architecture

Discussion

What is the difference between Macro-Architecture and Micro-Architecture?

Macro Software Architecture

Architecture dealing with large scale decisions

- concerning multiple software systems
- concerning multiple teams
- concerning organizational structure and its interplay with technology

- Examples:
  - Designing the overarching architecture for a software system developed by > 50 people
  - Designing architectural guidelines and rules for an entire company

Micro Software Architecture

Architecture dealing with small scale decisions

- concerning internal structure of a software system
- concerning a single team (sometimes even just one person)
- No to minor impact on organizational structure

- Examples:
  - Designing classes within a single program
  - Designing interfaces for a data access layer
  - Deciding to use library X for a single use-case

Your suggestions

Micro-Architecture or Macro-Architecture?

1. We use Maven to build this JAR.
2. All teams in our company use Java.
3. We are using Microservices.
4. We provide a central Apache Kafka message broker system for communication between our departments' software systems.
5. We use Flyway for data base migration scripts.

The Plan™

- The basics
  - Software Quality
  - Software Architecture Methods
  - Modeling
- Microarchitecture
  - Microarchitecture Principles
  - Design Patterns
- Macroarchitecture
  - Architectural Styles
  - Modern Architectures and Conway's Law
- Examples of Modern Architectures
  - Microservices
  - Cloud-native Systems
  - (CI/CD + DevOps)

Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski

TH Aschaffenburg
university of applied sciences